

A Web Platform for Generating Optimization-Ready V2V/V2I Connectivity Data from SUMO Vehicular Scenarios

Edison Xavier Espinel Sevilla ^{1,*} 

¹ Escuela Politécnica Nacional, Quito, Ecuador; edison.espinel@epn.edu.ec

Abstract

Roadside-unit (RSU) deployment models need data that tell which vehicles can reach each candidate RSU, in which mobility snapshot, and with how many hops. Producing these data is not trivial: it means putting together traffic mobility, urban geometry, direct-link evaluation, graph processing, and optimization-oriented data structures. This paper presents a web platform that turns a geographical area and a SUMO mobility scenario into an optimization-ready connectivity dataset. The backend integrates OpenStreetMap and SUMO tools, filters candidate RSU positions, evaluates binary V2V and V2I links from communication range and building obstruction, and computes cumulative and minimum-hop connectivity through a matrix recurrence. *A formal proposition shows that the recurrence finds every vehicle–RSU path within a given hop limit, while first-arrival matrices identify its minimum hop count. The resulting sparse tuple set lists each reachable real RSU at its minimum hop, plus an always-available artificial no-service option for every vehicle.* The Streamlit frontend supports scenario configuration, execution, visualization, and export. The platform is evaluated in a real urban area of the Historic Centre of Quito by varying the minimum junction degree and the spatial clustering radius used to build the candidate RSU set. **[TO DO: Insert the verified range of retained candidates and reduction percentages for the final parameter configurations.]** The workflow separates mobility simulation from infrastructure optimization, so the same generated dataset can be reused by different exact or heuristic deployment models.

Keywords: VANET; V2V connectivity; V2I connectivity; SUMO; multi-hop connectivity; roadside units; preprocessing; facility location; operations research

1. Introduction

Vehicular ad hoc networks (VANETs) enable direct vehicle-to-vehicle (V2V) communication and communication between vehicles and fixed roadside infrastructure (V2I). This infrastructure is usually a set of roadside units (RSUs) installed at selected locations, such as road intersections. Where these units are placed affects communication coverage, path length, infrastructure cost, and the operational performance of the network. For this reason, RSU deployment is often stated as a facility-location problem and solved with exact, heuristic, or metaheuristic methods.

Deployment models need connectivity information *before* the optimization starts. In particular, they need to know which vehicles can reach each candidate RSU in each mobility snapshot, and the minimum number of relay hops required. Generating this information is not a minor preprocessing step. It combines traffic simulation, extraction of vehicle positions, a representation of the urban geometry, communication-range checks, obstacle detection, graph processing, and conversion into the structures that an optimization solver

Published:

Copyright: © 2026 by the author.

Submitted to *Eng. Proc.* for possible open access publication under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](#) license.

accepts. Even so, many deployment studies treat these data as already available and describe their generation only briefly.

The software side tells a similar story. SUMO offers microscopic traffic simulation and tools to import OpenStreetMap (OSM) networks, while Python libraries provide geographical interfaces, data analysis, and visualization. These pieces are usually glued together with independent scripts, which hurts reproducibility and makes it hard to reuse the same scenario-generation process across alternative optimization formulations.

This work fills that gap with a web-based preprocessing layer that follows the sequence

urban area $\rightarrow$ SUMO snapshots $\rightarrow$ candidate RSUs $\rightarrow$ V2V/V2I matrices
 $\rightarrow$ minimum-hop tuples $\rightarrow$ optimization input.

The main output is not just a collection of connectivity matrices. It is a traceable dataset that keeps the relation between geographical positions, SUMO identifiers, matrix indices, mobility snapshots, hop counts, and optimization identifiers.

The contributions are:

1. a web platform that integrates a Streamlit frontend with a Python/SUMO backend in a reproducible workflow;
2. automatic generation of direct V2V and V2I connectivity matrices under a binary communication-range and line-of-sight (LoS) model;
3. a configurable two-stage method that reduces the number of candidate RSU positions using junction degree and spatial clustering;
4. computation of cumulative and minimum-hop vehicle–RSU connectivity matrices;
5. export of optimization-ready dense structures and sparse tuples, including an artificial no-service option for every vehicle;
6. a formal correctness result for the multi-hop recurrence and the minimum-hop representation;
7. a sensitivity analysis of how the minimum degree and the clustering radius change the number of retained RSU candidates.

The optimization model that picks the final deployment is out of scope here. The goal is to generate its input data automatically and independently of any specific formulation. The rest of the paper is organized as follows. Section 2 reviews RSU placement and SUMO-based tools. Section 3 presents the platform and the connectivity-data methodology. Section 4 reports the experimental setup and results. Section 5 concludes and lists future work.

2. Related Work

2.1. RSU Deployment and Facility–Location Models

RSU deployment has been studied through covering, median, routing, and stochastic formulations. Maximal-covering models try to cover the largest demand with a limited number of RSUs; set-covering models minimize how many units are needed to satisfy coverage; and median-type models reduce the average distance or hop count. Mobility-aware formulations extend these ideas over several traffic snapshots. The stochastic model in [1] uses scenario-dependent connectivity tuples and adds an artificial disconnected RSU as a no-service alternative that keeps the problem feasible when a vehicle cannot, or should not, be assigned to a deployed real RSU.

Despite their differences, all of these formulations rely on a common data layer: candidate RSUs, vehicle sets, mobility scenarios, and feasible vehicle–RSU relationships. This work focuses on producing exactly that layer. **[TO DO: Complete this subsection with re-**

cent studies on mobility-aware RSU placement, multi-hop access, and exact or heuristic deployment. Every added reference must be cited in the text.]

2.2. SUMO-Based Platforms and Web Interfaces

SUMO is an open-source microscopic traffic simulator with utilities to import OSM networks, generate vehicle trips, run simulations, and export floating-car data (FCD) [2,3]. Streamlit and Folium support interactive Python web interfaces and geographical visualization [4,5]. Their individual capabilities are not, by themselves, the contribution of this work. The contribution is their orchestration together with candidate filtering, geometric direct-link evaluation, multi-hop processing, identifier traceability, and export to optimization models.

Existing SUMO-based applications often focus on simulation control or visualization. This platform instead treats simulation as the first stage of a data-generation process whose final output can be consumed by facility-location and network-design formulations. [TO DO: Add a short comparison with the closest scientific platforms, considering mobility simulation, obstacle-aware V2V/V2I generation, multi-hop processing, web interaction, optimization export, and reproducibility.]

3. Web Platform and Connectivity-Data Generation

3.1. Platform Architecture and Workflow

The platform follows a simplified model–view–controller organization. The Streamlit frontend captures the geographical area and the configurable parameters, starts the processing workflow, and displays maps, matrices, and summary indicators. The Python backend prepares the SUMO scenario, extracts mobility snapshots, filters candidate RSUs, evaluates direct links, computes multi-hop reachability, and exports the resulting dataset. An orchestrator coordinates the modules and stores the configuration used in each run.

Figure 1 summarizes the architecture. The workflow starts when the user selects a geographical bounding box. OSM data are converted into a SUMO road network and building polygons with `netconvert` and `polyconvert`. Vehicle demand is generated with `randomTrips.py`, and the simulation produces FCD snapshots. The processing engine then evaluates direct V2V/V2I links, computes the multi-hop relations, and exports the optimization data.

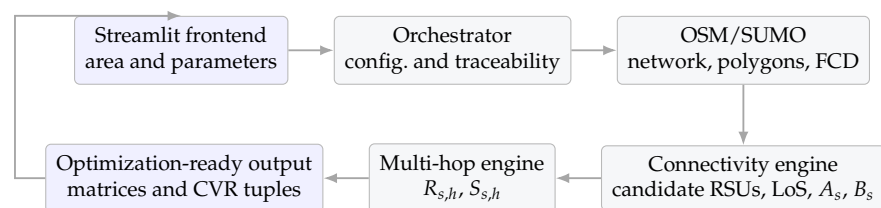


Figure 1. Platform architecture and end-to-end data-generation workflow.

The main configurable groups are: (i) mobility parameters, including the bounding box, simulation duration, vehicle-generation settings, snapshot interval, and random seed; (ii) candidate-selection parameters, that is, the minimum junction degree $g_{\min}$ and the clustering radius ρ ; (iii) communication parameters, namely the V2V and V2I ranges and whether building obstruction is used; and (iv) the maximum hop count H . [TO DO: Confirm the complete list and default values exposed by the current interface.]

3.2. Candidate RSU Selection

SUMO can produce a large set of junctions, including intermediate road points, internal nodes, and closely located intersections. Using all of them as deployment candidates

enlarges the optimization search space and adds nearly equivalent positions. The platform therefore applies a two-stage reduction.

First, it computes the topological degree of each non-internal junction and keeps only nodes whose degree is at least $g_{\min}$. Second, it sorts the surviving nodes by decreasing degree and runs greedy spatial clustering: a node becomes a candidate only when its distance to every previously accepted candidate is greater than ρ . Because higher-degree nodes are processed first, they represent their local groups.

Algorithm 1: Candidate RSU filtering by degree and greedy spatial clustering

Input: Junction set J , non-internal edge set E , minimum degree $g_{\min}$, clustering radius ρ

Output: Candidate RSU set C

```

1 Initialize  $\deg(j) \leftarrow 0$  for all  $j \in J$ ;
2 foreach  $(u, v) \in E$  do
3    $\lfloor$  increment  $\deg(u)$  and  $\deg(v)$ ;
4  $F \leftarrow \{j \in J : \deg(j) \geq g_{\min}\}$ ;
5 Sort  $F$  by non-increasing degree;
6  $C \leftarrow \emptyset$ ;
7 foreach  $j \in F$  do
8   if  $C = \emptyset$  or  $\min_{c \in C} \text{dist}(j, c) > \rho$  then
9      $\lfloor C \leftarrow C \cup \{j\}$ ;
10 return  $C$ ;
```

This procedure is a preprocessing heuristic, not an optimal RSU-placement algorithm. Its purpose is to build a manageable candidate set before the deployment model is solved. Since it may drop a location that would later be useful, the sensitivity of the retained set to $(g_{\min}, \rho)$ has to be reported instead of hiding these choices as fixed constants.

3.3. Direct and Multi-Hop Connectivity

For each snapshot $s \in \mathcal{S}$, let $\mathcal{V}_s = \{v_1, \dots, v_{n_s}\}$ be the active vehicles and $\mathcal{R} = \{r_1, \dots, r_m\}$ the candidate RSUs. The binary V2V matrix $A_s \in \{0, 1\}^{n_s \times n_s}$ has $(A_s)_{ij} = 1$ when vehicles v_i and v_j are within the V2V range and the segment joining them is not blocked by a building polygon. Under the assumed bidirectional-link model the matrix is symmetric. The binary V2I matrix $B_s \in \{0, 1\}^{n_s \times m}$ has $(B_s)_{ik} = 1$ when vehicle v_i can directly reach candidate RSU r_k under the V2I range and the LoS condition.

Building obstruction is tested with a two-dimensional segment–polygon intersection check. Candidate pairs are first filtered by communication range and by bounding boxes; the link segment is then tested against the relevant building edges. This yields a geometric binary approximation of connectivity. It does not model signal attenuation, fading, interference, or packet-delivery probability.

A vehicle without direct V2I access may still reach an RSU through other vehicles. Define the augmented adjacency matrix

$$\tilde{A}_s = \beta(A_s + I),$$

where $\beta(X)$ maps every positive entry of X to one and every zero entry to zero. Cumulative connectivity is

$$R_{s,1} = B_s, \tag{1}$$

$$R_{s,h} = \beta(\tilde{A}_s R_{s,h-1}), \quad h = 2, \dots, H. \tag{2}$$

The identity term keeps the paths already found at smaller hop counts, which gives $R_{s,1} \leq R_{s,2} \leq \dots \leq R_{s,H}$ elementwise. The first-arrival matrices are

$$S_{s,1} = R_{s,1}, \quad (3)$$

$$S_{s,h} = R_{s,h} - R_{s,h-1}, \quad h = 2, \dots, H. \quad (4)$$

Proposition 1. For any snapshot s , $(R_{s,h})_{ik} = 1$ if and only if there is a path from vehicle v_i to candidate RSU r_k using at most h communication hops, where the final hop is V2I and every previous hop is V2V.

Proof. The result follows by induction on h . For $h = 1$, $R_{s,1} = B_s$, so an entry is one exactly for a direct V2I link. Assume the statement holds for $h - 1$. The entry $(\tilde{A}_s R_{s,h-1})_{ik}$ is positive if there is an index j with $(\tilde{A}_s)_{ij} = 1$ and $(R_{s,h-1})_{jk} = 1$. If $j = i$, the identity term keeps a path of at most $h - 1$ hops. If $j \neq i$, then $(A_s)_{ij} = 1$ adds one V2V hop from v_i to v_j before a path of at most $h - 1$ hops from v_j to r_k . Conversely, every path of at most h hops either already has at most $h - 1$ hops or begins with one V2V hop followed by such a path. Binarization therefore gives exactly the stated reachability relation. $\square$

Corollary 1. For $h \geq 2$, $(S_{s,h})_{ik} = 1$ if and only if the minimum number of hops from v_i to r_k is exactly h . For $h = 1$, $S_{s,1} = B_s$ identifies the direct connections.

The corollary follows from Proposition 1 and the monotonicity of $R_{s,h}$: each reachable vehicle–RSU pair appears for the first time at a single hop value only. Figure 2 gives a compact example, where purple cells mark the first arrival at the corresponding hop.

R_3	r_1	r_2	r_3	S_2	r_1	r_2	r_3	S_3	r_1	r_2	r_3
v_1	1	1	1	v_1	0	0	1	v_1	0	1	0
v_2	1	1	1	v_2	1	1	0	v_2	0	0	0
v_3	1	1	1	v_3	0	0	1	v_3	1	0	0
v_4	0	0	0	v_4	0	0	0	v_4	0	0	0

Figure 2. Illustrative cumulative and first-arrival matrices for $H = 3$. Vehicle v_4 cannot reach any real RSU within the hop limit.

Equations (1)–(4) use ordinary dense matrix multiplication followed by thresholding. A Boolean-semiring implementation could reduce the average computational effort by avoiding unnecessary integer accumulation, but it does not change the dense worst-case order. Its implementation and comparison are left as future work.

3.4. Optimization-Oriented Export and Cardinality

Let

$$K_s = \sum_{h=1}^H \text{nnz}(S_{s,h}) = \text{nnz}(R_{s,H})$$

be the number of reachable real vehicle–RSU pairs in snapshot s . The export module creates one tuple for each real pair at its minimum hop count and one artificial no-service tuple for every active vehicle:

$$\text{CVR}_s = \{ \langle s, h, i, k \rangle : (S_{s,h})_{ik} = 1 \} \cup \{ \langle s, H + 1, i, r_\infty \rangle : i \in \mathcal{V}_s \}.$$

The artificial RSU r_∞ does not mean only that a vehicle is physically disconnected. It represents the optimization alternative of leaving that vehicle unserved because real candidates may not be deployed, may be saturated, or may be rejected due to cost or other

constraints. For this reason the tuple is included even when the same vehicle can reach one or more real RSUs.

The cardinality is

$$|\text{CVR}_s| = K_s + n_s,$$

and, for $m \geq 1$,

$$n_s \leq |\text{CVR}_s| \leq n_s(m + 1).$$

The factor H does not multiply the maximum number of real tuples, because each reachable pair (i, k) is exported once, at its minimum hop. Across all snapshots,

$$\sum_{s \in \mathcal{S}} n_s \leq |\text{CVR}| \leq (m + 1) \sum_{s \in \mathcal{S}} n_s.$$

The same information can be written to an OPL/.dat file or kept as in-memory structures for other solvers. **[TO DO: Confirm the exact final tuple-field names, the artificial-RSU identifier, the penalty convention, and the repository example used by the optimization model.]**

4. Experimental Results

4.1. Experimental Settings

The evaluation uses one real scenario in the Historic Centre of Quito, Ecuador. The goal is not to compare different cities, but to measure how the candidate-generation parameters change the size of the optimization search space while the road network stays fixed. The two factors are the minimum junction degree $g_{\min}$ and the greedy clustering radius ρ .

The original manuscript reports 620 SUMO junctions and a preliminary filtered set of 160 candidates for one configuration, i.e. an approximate 74% reduction. These values are kept as preliminary evidence and must be reproduced from the final code before submission. The sensitivity analysis uses a compact set of configurations, such as those in Table 1; the exact values may be adjusted to match the implemented interface.

Table 1. Proposed parameter configurations for candidate-set sensitivity.

Configuration	$g_{\min}$	ρ (m)	Purpose
A	3	15	permissive degree, local clustering
B	3	20	effect of a larger radius
C	4	20	reference configuration
D	5	25	restrictive degree, wider clustering

For every configuration, the experiment reports the original number of eligible SUMO junctions X , the number of retained candidates Y , and the reduction

$$\text{Reduction}(\%) = 100 \left(1 - \frac{Y}{X} \right).$$

The runs must use the same downloaded network and the same rules for excluding internal nodes, which isolates the effect of $(g_{\min}, \rho)$. **[TO DO: Record the map source date, the geographical bounding box, the SUMO version, the Python version, the hardware, and the exact random seed.]**

4.2. Results

Table 2 is the final reporting structure. It deliberately leaves unverified measurements in red instead of filling them with invented values.

Table 2. Candidate-set sensitivity in the Historic Centre scenario.

Config.	$g_{\min}$	ρ (m)	Original X	Retained Y	Reduction (%)
A	3	15	[TO DO: measure]	[TO DO: measure]	[TO DO: compute]
B	3	20	[TO DO: measure]	[TO DO: measure]	[TO DO: compute]
C	4	20	620*	160*	74.2*
D	5	25	[TO DO: measure]	[TO DO: measure]	[TO DO: compute]

*Preliminary value inherited from the original manuscript; final reproduction required.

The expected monotonic tendencies are clear, but should not be reported as measured findings until the runs are done. Raising $g_{\min}$ removes low-degree junctions before clustering. Raising ρ suppresses candidates located close to a previously retained higher-degree node. Stronger values therefore normally reduce Y , although the marginal effect depends on the spatial distribution and the degree structure of the fixed road network.

A compact final figure should plot the retained candidates and the reduction percentage against the four configurations. A second map may compare the least and the most restrictive configurations if it clearly reveals the spatial effect without repeating the architecture figure. **[TO DO: Insert the final plot(s) generated from the verified sensitivity table.]**

The preliminary reference configuration reduces the candidate set from 620 to 160 positions. If reproduced, this means the deployment model considers about one quarter of the original junctions. That matters in practice, because binary deployment variables and many assignment or routing structures grow with the number of candidate facilities. Still, candidate reduction is not the same as optimization quality: a smaller candidate set can speed up the model while excluding useful locations. The final paper should therefore describe the reduction as a preprocessing trade-off, not as proof of a better deployment.

The platform also completed the end-to-end transformation from SUMO snapshots to matrix and tuple outputs in the supplied case study. The source manuscript reports 132 vehicles, 31 snapshots, and 1710 connectivity tuples, but these figures need a consistent re-run, because one reported RSU-related range was inconsistent with the filtered candidate count. **[TO DO: Verify the number of vehicles, snapshots, real tuples, artificial tuples, and total |CVR| under the final configuration.]**

The current implementation has three main limitations. First, its binary geometric link model ignores path loss, multipath fading, interference, channel load, and packet-level reliability. Second, every snapshot is processed independently, so the recurrence describes paths *inside* a snapshot and not time-respecting store-carry-forward routes. Third, candidate filtering is heuristic, and its effect on the objective value of a later deployment model has not been quantified yet. These limitations set the meaning of the generated data and motivate the future work below.

5. Conclusions and Future Work

This paper presented a web platform that turns a geographical area and SUMO mobility output into optimization-ready V2V/V2I connectivity data. The workflow puts together candidate RSU generation, direct-link evaluation with building obstruction, cumulative and minimum-hop matrix processing, traceable identifiers, visualization, and export. The formal result shows that the recurrence finds every vehicle–RSU relation reachable within the hop limit, while the first-arrival matrices assign each real pair its minimum hop count.

The exported sparse set contains all reachable real options and one artificial no-service option per vehicle. This distinction matters, because the artificial RSU stands for

an optimization decision, not only physical disconnection. The resulting cardinality is $K_s + n_s$, bounded by n_s and $n_s(m + 1)$ for each snapshot.

The experimental design focuses on one urban scenario and looks at how sensitive the candidate reduction is to the minimum junction degree and the clustering radius. Preliminary values point to a substantial reduction in the Historic Centre of Quito, but the final paper must replace the inherited figures with reproducible measurements and report the parameter-dependent trade-off explicitly.

Future work includes comparing the recurrence against breadth-first-search implementations, testing sparse and Boolean-semiring matrix kernels, and measuring when the connectivity matrices become dense. Another direction is to generate feasible tuples dynamically inside the optimization method, for example through decomposition or column generation, instead of materializing the whole preprocessing output. The communication model should also gain propagation and reliability information, and the influence of candidate filtering on final deployment cost, coverage, capacity, and solution time should be validated.

Author Contributions: Conceptualization, methodology, software, validation, writing—original draft preparation, and writing—review and editing, E.X.E.S. **[TO DO: Update if additional authors or contributors are included.]**

Funding: **[TO DO: Insert the SmartCityNet project name, grant identifier, and funding institution, or state that the research received no external funding.]**

Data Availability Statement: **[TO DO: Insert the final code repository and dataset link, together with the configuration files needed to reproduce the Historic Centre scenario.]**

Acknowledgments: **[TO DO: Insert the institutional acknowledgments and verify the venue policy for disclosure of AI-assisted editing.]**

Conflicts of Interest: The author declares no conflict of interest.

Abbreviations

The following abbreviations are used in this manuscript:

VANET	Vehicular Ad hoc Network
V2V	Vehicle-to-Vehicle
V2I	Vehicle-to-Infrastructure
RSU	Road Side Unit
OBU	On Board Unit
LoS	Line of Sight
FCD	Floating Car Data
CVR	Connectivity tuple set $\langle s, h, i, k \rangle$
SUMO	Simulation of Urban MObility
OSM	OpenStreetMap

References

1. Urquiza-Aguiar, L.; Tripp-Barba, C.; Aguilar-Igartua, M. A Stochastic Optimization Model for the Placement of Road Side Units. In *Proceedings of ADHOC-NOW*; Springer: Cham, Switzerland, 2016.
2. Lopez, P.A.; Behrisch, M.; Bieker-Walz, L.; et al. Microscopic Traffic Simulation using SUMO. In *Proceedings of the 21st IEEE International Conference on Intelligent Transportation Systems*; IEEE: Maui, HI, USA, 2018; pp. 2575–2582.
3. OpenStreetMap contributors. OpenStreetMap. Available online: <https://www.openstreetmap.org> (accessed on 20 July 2026).
4. Streamlit Inc. Streamlit. Available online: <https://streamlit.io> (accessed on 20 July 2026).

5. Python Visualization. Folium: Python Data, Leaflet.js Maps. Available online: <https://python-visualization.github.io/folium/> (accessed on 20 July 2026). 300
301

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are 302
solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). 303
MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting 304
from any ideas, methods, instructions or products referred to in the content. 305